COMP2007: Operating Systems & Concurrency
Week 11 – 12:00pm Monday – 02 December 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Figure: Attendance Monitoring

# Operating Systems and Concurrency

## File Systems 4

## COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

# Overview

## Context of File Systems

1. **Devices** and hard drives (rotational and SSD)
2. **Structures** (partitions, directories, FCBs, system calls)
3. Storing **data** “in” FCBs
   - **Contiguous implementations** are easy, fast, but result in external fragmentation
   - **Linked lists** are sequential (block sizes $\neq 2^n$)
   - **FAT** have block sizes $= 2^n$, but the table becomes prohibitively large
   - **i-nodes** are only loaded when the file is open, contain attributes and multiple block levels
4. High level **paradigms**

# Goals for Today

## Overview

- **Log structured** file systems
- **Journaling** file systems
- The Unix/Linux **virtual file system**

# Traditional File Systems

## Context

- Consider the **creation** of a **new file** on a Unix system:
  1. **Allocate**, **initialise** and **write** the **i-node** for the file
     - i-nodes are “usually” located at the **start of the disk**
  2. **Update and write the directory** entry for the file
  3. **Write the data** to the disk
- Data, i-nodes, directories are not necessarily in **adjacent locations**
- **Random disk access** is slow(er) on traditional **rotational disks** and **SSDs**

# Traditional File Systems

## Creating and Writing /foo/bar (based on Arpaci-Dusseau)

### Creating and writing a file

Assume we are writing to `/foo/bar`

| | | data bitmap | inode bitmap | root inode | foo inode | bar inode | root data | foo data | bar[0] data | bar[1] data | bar[2] data |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | | | | read | | | | | | | |
| 2 | | | | | | | read | | | | |
| 3 | | | | | read | | | | | | |
| 4 | | | | | | | | read | | | |
| 4 | create() | | read | | | | | | | | |
| 5 | | | write | | | | | | | | |
| 6 | | | | | | | | write | | | |
| 7 | | | | | | read | | | | | |
| 8 | | | | | | write | | | | | |
| 9 | | | | | write | | | | | | |
| 11 | | | | | | read | | | | | |
| 12 | | read | | | | | | | | | |
| 13 | write() | write | | | | | | | | | |
| 14 | | | | | | | | | write | | |
| 15 | | | | | | write | | | | | |

# Log Structured File System

## Concept

- The file system is treated as a **circular log**
- **Read and write operations** (i-nodes, data, etc.) are **buffered in memory**
  - Once full, the buffer is flushed to the disk as **one contiguous segment** at the head of "**the log**"
  - **Data, directories and i-nodes** are all written to the **same "segment"**
  - An **i-node map** is maintained in memory to quickly find the address of i-nodes on the disk
- A **cleaner thread** runs the background, scans the tail of the log, and **compacts** it
  - Files that are in use are written to the **"head" of the log**
  - **Deleted** files are "reclaimed

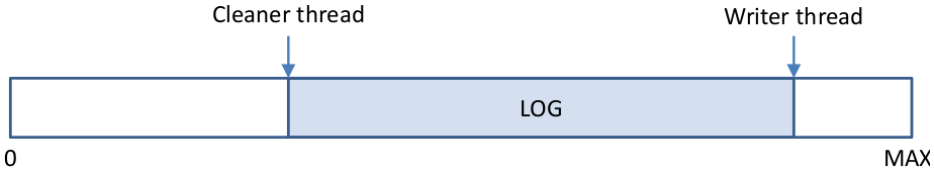
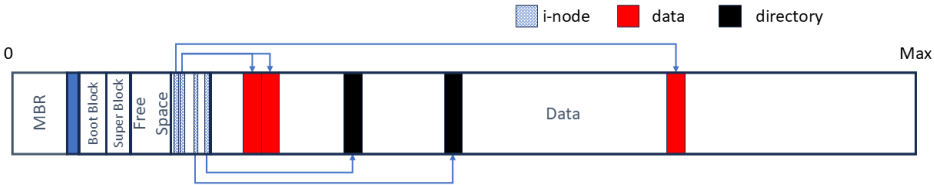



Figure: Log Structured File System

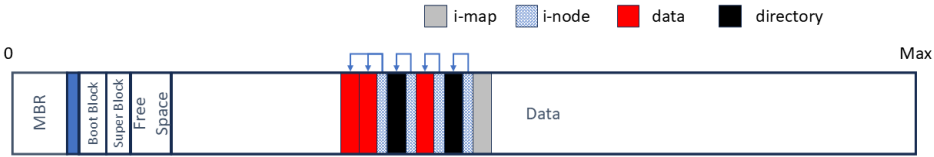
# Log Structured File System

## Structure



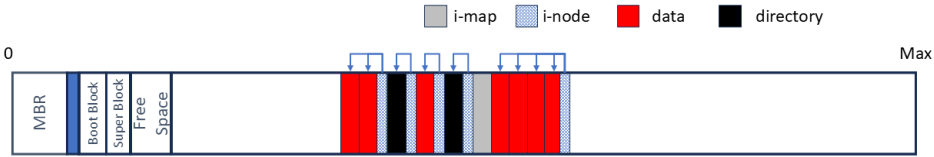
# Log Structured File System

## Structure



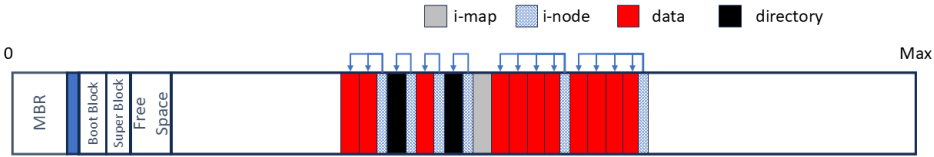
# Log Structured File System

## Structure



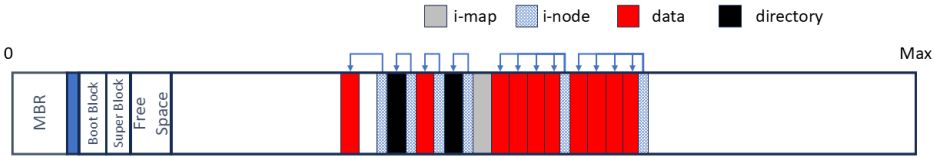
# Log Structured File System

## Structure



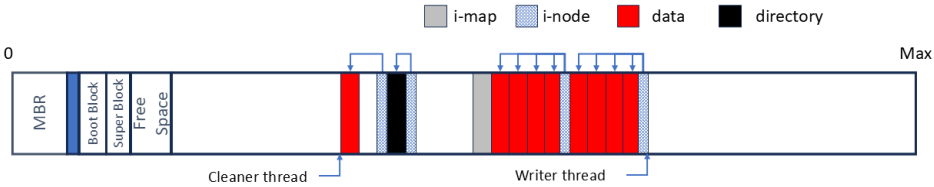
# Log Structured File System

## Structure



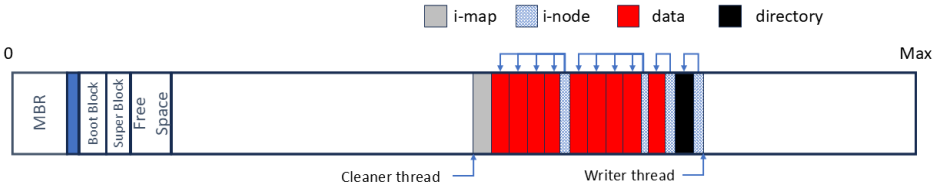
# Log Structured File System

## Structure



# Log Structured File System

## Structure



# Log Structured File System

## Advantages and Disadvantages

- It greatly **increases disk performance** but the **cleaner thread** requires additional CPU time
- Writes are **more robust** as they are done as a single operation (multiple small writes are more likely to expose the file system to serious inconsistency)

# Log Structured File System

## Applications

### Applications

Any contexts in which log structured file system may be useful?

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

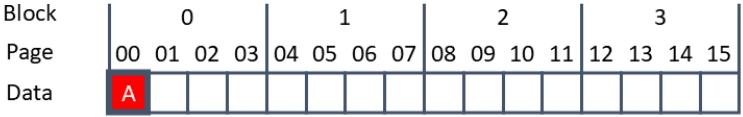



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

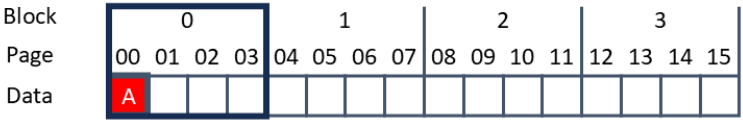



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

| Block | 0 | | | |
|---|---|---|---|---|
| Page | 00 | 01 | 10 | 11 |
| Data | 00101001 | 10100101 | 11101011 | 00001010 |

Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

| Block | 0 | | | |
|---|---|---|---|---|
| Page | 00 | 01 | 10 | 11 |
| Data | 11111111 | 11111111 | 11111111 | 11111111 |

Figure: SSD Write Operation

©University of Nottingham

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

| Block | 0 | | | |
|---|---|---|---|---|
| Page | 00 | 01 | 10 | 11 |
| Data | 10101010 | 11111111 | 11111111 | 11111111 |

Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

| Block | 0 | | | |
|---|---|---|---|---|
| Page | 00 | 01 | 10 | 11 |
| Data | 10101010 | 10100101 | 11101011 | 00001010 |

Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

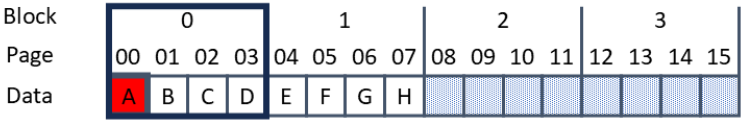



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

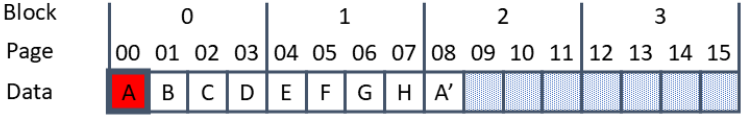



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

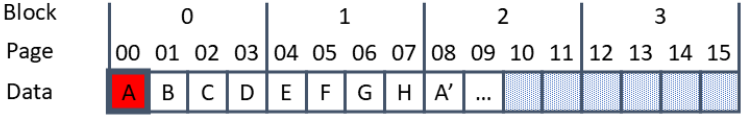



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

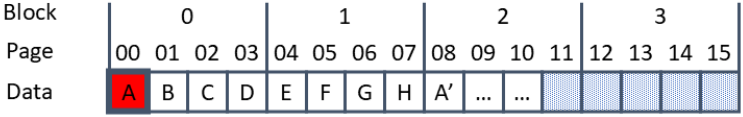



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

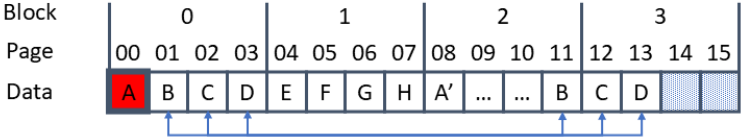



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSDs

**SSDs** suffer from **write amplification** (block must be erased), **write disturbance**, and **wear out**

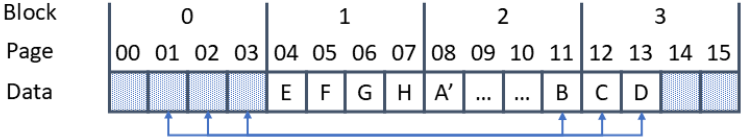



Figure: SSD Write Operation

# Log Structured File System

## SSDs

### SSD Log Structured File System

- **Log structured file systems** are typically implemented in the **SSD controller**. Any problems with this?

# Log Structured File System

## SSDs

### SSD Log Structured File System

- **Log structured file systems** are typically implemented in the **SSD controller**. Any problems with this?
- The **SSD changes the block addresses** where data is stored and **does not have access to the FCBs** / **i-nodes**. Any solutions?

# Log Structured File System

## SSDs

### SSD Log Structured File System

- **Log structured file systems** are typically implemented in the **SSD controller**. Any problems with this?
- The **SSD changes the block addresses** where data is stored and **does not have access to the FCBs** / **i-nodes**. Any solutions?
- A **logical address space** in the file system is mapped onto a **physical address space** in the SSD using a **mapping table**

©University of Nottingham

# Journaling File System

## Example

- **Deleting a file** consists of the following actions:
  1. **Remove** the file's **directory** entry
  2. Add the file's **i-node** to the **pool of free i-nodes**
  3. Add the file's **data blocks** to the **free list**
- It can **go wrong**, for instance:
  - **Directory** entry has been **deleted** and a crash occurs ⇒ i-nodes and disk blocks become inaccessible
  - The **directory** entry and **i-node** have been released and a crash occurs ⇒ disk blocks become inaccessible

# Journaling File System

## Example (Cont'ed)

- **Changing the order** of the events does not necessarily resolve the issues
- Journaling file systems aim at **increasing the resilience** of file systems against **crashes** by recording each update to the file system as a transaction

# Journaling File System

## Concept

- Journaling file systems **log all events** (transactions) before they take place
  1. Write the actions that should be undertaken to a log file
  2. Carry them out
  3. Remove/commit the entries once completed
- The **log is examined** after a **crash** and any **remaining entries are actioned**
- **NTFS**, **ext3**, and **ext4** and are examples of journaling file systems

# Virtual File System

## Concept

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class Demo1 {
  public static void main(String[] args) throws IOException {
    FileWriter fw =
      new FileWriter(("C:/Program Files (x86)/test.txt"));
    PrintWriter pw = new PrintWriter(fw);
    pw.close();
  }
}
```

# Virtual File System

## Examples

```
$ df -Th
Filesystem    Type        Size  Used Avail Use% Mounted on
devtmpfs      devtmpfs     32G     0   32G   0% /dev
/dev/sda3     btrfs       215G   82G  132G  39% /
tmpfs         tmpfs        32G  8.6M   32G   1% /tmp
/dev/sda3     btrfs       215G   82G  132G  39% /home
/dev/sda1     ext4        477M  166M  282M  38% /boot
/dev/sdb      ext4        917G  165G  706G  19% /data
pszgd@bann:   fuse.sshfs  700M  282M  418M  41% /mnt/bann
```

©University of Nottingham

# Virtual File System

## Concept

- **Multiple file systems** usually co-exist (e.g., NTFS and ISO9660 for a CD-ROM, NFS)
- File systems are **seamlessly integrated** by the operating system's **virtual file systems** (VFS)
- VFS relies on **standard "object oriented"** principles (or manual implementations thereof), e.g. polymorphism

# Virtual File System

## Concept

- Consider code to **read "data records"** (`DataObject`) from a file
- These records can be stored in a **CSV file** or an **XML File**

### Maintainable Code

How would you make your code resilient against **changes in the underlying data structure**?

# Virtual File System

## Concept

- Consider code to **read "data records"** (`DataObject`) from a file
- These records can be stored in a **CSV file** or an **XML File**

### Maintainable Code

How would you make your code resilient against **changes in the underlying data structure**?

⇒ You would hide the **implementation** behind an **interface** using **Data Access Objects** (DAOs)

# Virtual File System

## Concept

- We can define a **generic interface**, e.g. `DataReader`, containing a method `public DataObject readData();`
- You would hide the CSV and XML code in **specific implementations** of the `DataReader` interface, e.g. `CSVDataReader` and `XMLDataReader`

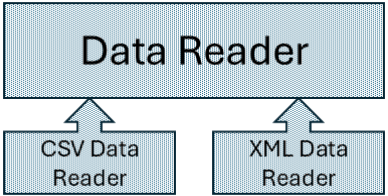



Figure: Data Reader

# Virtual File System

## Concept

```
public class ClientApplication { // imports above
  public static void main(String[] args) throws IOException {
    System.out.println("Choose CSV or XML?");
    BufferedReader br
        = new BufferedReader(new InputStreamReader(System.in));
    String type = br.readLine();
    br.close();

    DataReader reader = null;
    if (type.equals("CSV")) {
      reader = new CSVDataReader();
    } else if (type.equals("XML")) {
      reader = new XMLDataReader();
    }

    if (reader != null)
      System.out.println(reader.readData());
  }
}
```

©University of Nottingham

# Virtual File System

## Concept

- In a similar way, Unix and Linux **unify different file systems** using the `posix` interface
- File system specific code is in the **implementation layer** that is **separated from the interface**

# Virtual File System

## Concept

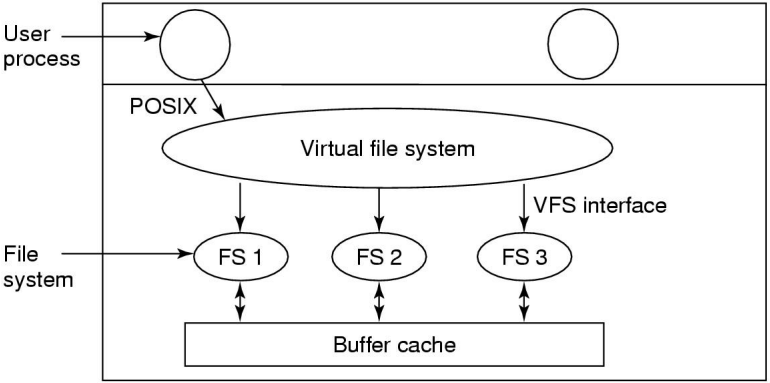



Figure: Virtual File System (Tanenbaum)

# Virtual File System

## Concept

- The VFS interface commonly contains the **POSIX system calls** (`open`, `close`, `read`, `write`, ...)
- Each file system that meets the VFS requirements **provides an implementation** for the system calls contained in the interface
- Note that implementations can be for **remote file systems**

# Virtual File System

## In practice

- Every file system, including the root file system, is **registered with the VFS**
  - A list / table of addresses to the **VFS function calls** (i.e. function pointers) for the specific file system is provided
  - **Every VFS function call** corresponds to a specific **entry in the VFS function table** for the given file system
  - The **VFS maps** / **translates** the POSIX call onto the **"native" file system call**
- A virtual file system is essentially **good programming practice**

©University of Nottingham

# Summary

## Take-Home Message

- File system **paradigms**:
  - Logs: store everything as close as possible
  - Journaling: apply the transaction principle
- VFS: apply good software design (polymorphism)